Week 7 Submission

Peer-to-Peer Lending Matchmaking System

Regd No	25091A32G9, 25091A32K4
Problem Statement No	105
Title	Peer-to-Peer Lending Matchmaking System
Technology	Python, Flask, SQLite, NetworkX, Pandas/NumPy, ReportLab, OpenPyXL, Chart.js, Bootstrap, unittest/pytest
Data	Academic simulation using simulated test data - no real-money transactions

Where to run the code (read this first)

- **Where:** on your own laptop. Install **Python 3.10 or newer** from python.org (tick *Add Python to PATH*) and **VS Code**. Everything below is typed in the **VS Code terminal** (menu Terminal > New Terminal, or Ctrl+`).
- **Which folder:** open your **p2p_lending** folder (the one that contains **app.py**) with File > Open Folder. Every command must be run from this folder, not from a sub-folder, not in IDLE.
- **One-time setup:** run `python -m venv venv`, then activate it - Windows: `venv\Scripts\activate` (if PowerShell blocks it use Command Prompt or run `Set-ExecutionPolicy -Scope Process Bypass`), macOS/Linux: `source venv/bin/activate`. Then `pip install -r requirements.txt`.
- **Adding the files of a week:** in VS Code right-click the folder > New File, type the exact path shown in the heading (for example `routes/profiles.py`), paste the code. **[NEW]** = create the file, **[REPLACE]** = overwrite the old file completely, **[APPEND]** = paste at the very end of the existing file.
- **Web pages:** run `python app.py`, then open `http://127.0.0.1:5000` in your browser (stop the server with Ctrl+C). The pages load Bootstrap/Chart.js from the internet, so stay online.
- **Keep the weeks cumulative:** Week N needs the files of Weeks 1 to N-1. Your Week 1 zip is the starting point and none of its files is deleted later (only app.py and templates/index.html are replaced in Week 2, and config.py gets lines appended in Week 3).
- **Demo scripts** (`demo_weekN.py`) create their own throw-away databases in the data folder, so they never disturb your real database. To see the same demo data in the web pages: stop the server, delete `data/p2p_lending.db`, run `python demo_data.py`, start the server again.

Week 7: Testing and Delivery

Test scale, edge cases, scoring and reports; evaluate the system; prepare source code, database, sample dataset, test report, screenshots, documentation and the demonstration. **Output: complete tested P2P lending simulation system.**

Assignment steps and where they are done

Assignment step	Implemented in
Test 1: multiple lenders competing for one borrower	test_week7.py: test_01 (best-fitting lender wins, others expire)
Test 2: one lender matched with multiple loans	test_02
Test 3: no compatible lender	test_03
Test 4: loan amount exceeding available funds	test_04
Test 5: risk appetite mismatch	test_05
Test 6: repayment-term mismatch	test_06
Test 7: duplicate loan requests	test_07
Test 8: large lender and borrower datasets	test_08 (60 lenders x 100 borrowers) and evaluate.py (up to 200 x 300)
Test 9: paid, late and defaulted repayment cases	test_09
Test 10: incorrect or missing profile information	test_10
Evaluate: execution time, eligible matches, allocation success rate, constraint violations, explanation correctness, database and audit consistency	evaluate.py (prints every metric and writes test_report.md)

Files of this week

File	Action	Purpose
generate_data.py	NEW	Creates simulated lenders, borrowers and requests as CSV files and/or database rows
evaluate.py	NEW	Runs the full pipeline and measures all required metrics
tests/test_week7.py	NEW	The 10 required scenarios + 3 evaluation tests
README.md	NEW	Project documentation

What evaluate.py measures

Metric	How it is measured
Match execution time	wall-clock time of scoring + graph building + allocation
Number of eligible matches	edges in the bipartite graph
Match-allocation success rate	fully matched requests / pending requests
Constraint violations	every selected match is re-checked against the 7 hard rules, lender capacity and request amount (must be 0)
Explanation correctness	every stored explanation is compared with a fresh evaluation of the same pair (must be 100%)
Database and audit consistency	8 checks: audit entry for every loan and repayment change, schedule length, balances, principal totals, no negative funds, accepted matches = loans

How to run and check this week

1. `python -m pytest -v` - runs all 85 tests of Weeks 1-7 (or: python -m unittest discover -s tests -v). The full run takes about one minute.

2. `python generate_data.py --lenders 50 --borrowers 80 --out data/sample` - writes the sample dataset CSV files

3. `python evaluate.py --report test_report.md` - prints the metrics for 3 sizes and writes the test report

4. `python app.py` - final demonstration in the browser

Test summary (Weeks 1-7)

```
Ran 10 tests in 0.997s OK
Ran 14 tests in 4.008s OK
Ran 20 tests in 7.059s OK
Ran 10 tests in 3.835s OK
Ran 14 tests in 4.435s OK
Ran 4 tests in 1.657s OK
```

Expected output of evaluate.py

```
=== 10 lenders x 15 borrowers (21 requests, 210 pairs) ===
  match_seconds              0.015
  eligible_matches           52
  selected_matches           14
  requests_matched           14
  allocation_success_rate    66.7
  constraint_violations      0
  explanation_correct        210/210
  consistency_passed         8/8
  loans_created              14
  audit_entries              468
  consistency problems       none

=== 50 lenders x 80 borrowers (105 requests, 5,250 pairs) ===
  match_seconds              0.167
  eligible_matches           1608
  selected_matches           85
  requests_matched           85
  allocation_success_rate    81.0
  constraint_violations      0
  explanation_correct        5250/5250
  consistency_passed         8/8
  loans_created              85
  audit_entries              2979
  consistency problems       none

=== 200 lenders x 300 borrowers (385 requests, 77,000 pairs) ===
  match_seconds              2.394
  eligible_matches           21179
  selected_matches           297
  requests_matched           297
  allocation_success_rate    77.1
  constraint_violations      0
  explanation_correct        77000/77000
  consistency_passed         8/8
  loans_created              297
  audit_entries              11099
  consistency problems       none

Report written to test_report.md
```

Generated test_report.md

```
# P2P Lending Matchmaking - Evaluation Report

Funding mode: full funding.  Simulated data only.

| Lenders | Requests | Pairs | Match time (s) | Eligible | Selected | Matched | Success % | Violations | Explanations | Consistency |
|---|---|---|---|---|---|---|---|---|---|---|
| 10 | 21 | 210 | 0.015 | 52 | 14 | 14 | 66.7 | 0 | 210/210 | 8/8 |
| 50 | 105 | 5250 | 0.167 | 1608 | 85 | 85 | 81.0 | 0 | 5250/5250 | 8/8 |
| 200 | 385 | 77000 | 2.394 | 21179 | 297 | 297 | 77.1 | 0 | 77000/77000 | 8/8 |

## Consistency checks (largest run)

- [x] every loan has a LOAN_CREATED audit entry
- [x] every loan has one instalment per month
- [x] outstanding balance matches settled instalments
- [x] funded requests: loan principals add up to the request amount
- [x] no lender has negative available funds
- [x] accepted matches == loans
- [x] every repayment status change is in the audit log
- [x] funded requests have loans
```

[NEW] generate_data.py

```
"""Week 7 - generate a large SIMULATED dataset (CSV files and/or database rows).

    python generate_data.py --lenders 50 --borrowers 80 --out data/sample
    python generate_data.py --lenders 50 --borrowers 80 --load         # into data/p2p_lending.db
"""
import argparse
import os
import random

import pandas as pd

import database
import profile_service as ps
from models import ValidationError

PASSWORD = "password123"


def generate(n_lenders, n_borrowers, seed=42):
    rng = random.Random(seed)
    lenders, borrowers, requests = [], [], []
    for i in range(1, n_lenders + 1):
        min_loan = rng.choice([5000, 10000, 20000])
        min_return = round(rng.uniform(6, 11) * 2) / 2
        lenders.append({
            "full_name": f"Lender {i:04d}", "email": f"lender{i:04d}@sim.test", "password": PASSWORD,
            "risk_appetite": rng.choices(["low", "medium", "high"], weights=[3, 4, 3])[0],
            "available_amount": rng.randrange(100000, 2000001, 10000), "min_loan_amount": min_loan,
            "max_loan_amount": rng.choice([150000, 300000, 600000]),
            "min_duration_months": rng.choice([3, 6, 12]), "max_duration_months": rng.choice([24, 36, 48, 60]),
            "min_expected_return": min_return,
            "max_expected_return": min(30, min_return + round(rng.uniform(6, 14) * 2) / 2)})
    for i in range(1, n_borrowers + 1):
        income = rng.randrange(20000, 150001, 1000)
        borrowers.append({
            "full_name": f"Borrower {i:04d}", "email": f"borrower{i:04d}@sim.test", "password": PASSWORD,
            "monthly_income": income, "existing_obligations": round(income * rng.uniform(0, 0.5), -2),
            "credit_score": rng.randint(400, 850), "credit_history_years": round(rng.uniform(0.5, 15), 1)})
        used = set()
        for _ in range(rng.choice([1, 1, 1, 2])):
            amount, term = rng.randrange(10000, 400001, 1000), rng.choice([6, 12, 18, 24, 36, 48])
            if (amount, term) not in used:
                used.add((amount, term))
                requests.append({"borrower_index": i, "amount": amount, "term_months": term,
                                 "purpose": rng.choice(["Education", "Medical", "Business", "Home", "Vehicle"])})
    return lenders, borrowers, requests


def write_csv(out_dir, lenders, borrowers, requests):
    os.makedirs(out_dir, exist_ok=True)
    pd.DataFrame(lenders).drop(columns=["password"]).to_csv(os.path.join(out_dir, "sample_lenders.csv"), index=False)
    pd.DataFrame(borrowers).drop(columns=["password"]).to_csv(os.path.join(out_dir, "sample_borrowers.csv"), index=False)
    pd.DataFrame(requests).to_csv(os.path.join(out_dir, "sample_requests.csv"), index=False)


def load(conn, lenders, borrowers, requests):
    """Insert through the validated services.  Returns (lender_ids, borrower_ids, request_ids, skipped)."""
    lender_ids = [ps.register_lender(conn, d, auto_activate=True) for d in lenders]
    borrower_ids = [ps.register_borrower(conn, d, auto_activate=True) for d in borrowers]
    request_ids, skipped = [], 0
    for r in requests:
        try:
            request_ids.append(ps.create_loan_request(conn, borrower_ids[r["borrower_index"] - 1], r))
        except ValidationError:
            skipped += 1
    return lender_ids, borrower_ids, request_ids, skipped


def main():
    parser = argparse.ArgumentParser(description="Generate simulated P2P lending data")
    parser.add_argument("--lenders", type=int, default=50)
    parser.add_argument("--borrowers", type=int, default=80)
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--out", default=os.path.join("data", "sample"), help="folder for the CSV files")
    parser.add_argument("--load", action="store_true", help="also load the data into the application database")
    args = parser.parse_args()
    lenders, borrowers, requests = generate(args.lenders, args.borrowers, args.seed)
    write_csv(args.out, lenders, borrowers, requests)
    print(f"CSV files written to {args.out}: {len(lenders)} lenders, {len(borrowers)} borrowers, {len(requests)} requests")
    if args.load:
        database.init_db()
        conn = database.get_connection()
        _, _, request_ids, skipped = load(conn, lenders, borrowers, requests)
        conn.close()
        print(f"Loaded into database: {len(request_ids)} requests ({skipped} skipped)")


if __name__ == "__main__":
    main()
```

[NEW] evaluate.py

```
"""Week 7 - evaluation harness: runs the whole pipeline on simulated data and reports

   match execution time, number of eligible matches, match-allocation success rate,
   constraint violations, explanation correctness, database and audit consistency.

    python evaluate.py                                  # sizes 10x15, 50x80, 200x300
    python evaluate.py --sizes 20x30,100x150 --report test_report.md
"""
import argparse
import json
import os
import tempfile
import time

import database
import generate_data
import loan_service as ls
import profile_service as ps
import scoring
from matching_engine import MatchingEngine


def _risk_result(conn, request_id):
    row = conn.execute("SELECT * FROM risk_scores WHERE request_id = ?", (request_id,)).fetchone()
    import config
    return scoring.RiskResult(row["score"], row["risk_category"], json.loads(row["components"]),
                              row["explanation"], config.RATE_BY_RISK[row["risk_category"]])


def count_constraint_violations(conn, allow_partial):
    """Re-check every selected match against the hard rules, plus capacity limits."""
    checker = scoring.CompatibilityScorer(allow_partial)
    violations = 0
    rows = conn.execute("SELECT * FROM matches WHERE status = 'recommended' AND allocated_amount > 0").fetchall()
    for m in rows:
        lender = ps.get_lender(conn, m["lender_id"])
        request = ps.get_request(conn, m["request_id"])
        borrower = ps.get_borrower(conn, request.borrower_id)
        violations += len(checker.check_constraints(lender, borrower, request, _risk_result(conn, m["request_id"])))
    for row in conn.execute("SELECT SUM(m.allocated_amount) AS used, l.available_amount FROM matches m "
                            "JOIN lenders l ON l.lender_id = m.lender_id WHERE m.status = 'recommended' "
                            "AND m.allocated_amount > 0 GROUP BY m.lender_id"):
        violations += 1 if row["used"] > row["available_amount"] + 0.01 else 0
    for row in conn.execute("SELECT SUM(m.allocated_amount) AS got, r.amount FROM matches m JOIN loan_requests r "
                            "ON r.request_id = m.request_id WHERE m.status = 'recommended' AND m.allocated_amount > 0 "
                            "GROUP BY m.request_id"):
        violations += 1 if row["got"] > row["amount"] + 0.01 else 0
    return violations, len(rows)


def explanation_correctness(conn, allow_partial):
    """Fraction of stored explanations that agree with a fresh evaluation of the same pair."""
    checker, ok, total = scoring.CompatibilityScorer(allow_partial), 0, 0
    lenders = {l.lender_id: l for l in ps.list_lenders(conn)}
    cache = {}
    for m in conn.execute("SELECT * FROM matches"):
        total += 1
        if m["request_id"] not in cache:
            request = ps.get_request(conn, m["request_id"])
            cache[m["request_id"]] = (request, ps.get_borrower(conn, request.borrower_id),
                                      _risk_result(conn, m["request_id"]))
        request, borrower, risk = cache[m["request_id"]]
        result = checker.evaluate(lenders[m["lender_id"]], borrower, request, risk)
        text = m["explanation"]
        if m["status"] == "rejected":
            good = (not result.eligible) and text.startswith("REJECTED") and all(msg in text for _, msg in result.violations)
        elif m["status"] == "recommended":
            good = result.eligible and "SELECTED (rank" in text and abs(result.score - m["compatibility_score"]) < 0.01
        else:
            good = result.eligible and "NOT SELECTED" in text
        ok += good
    return ok, total


def consistency_checks(conn):
    """Database + audit consistency after loans were created and repayments simulated."""
    checks = {}
    loans = conn.execute("SELECT * FROM loans").fetchall()
    checks["every loan has a LOAN_CREATED audit entry"] = all(conn.execute(
        "SELECT COUNT(*) FROM audit_logs WHERE action='LOAN_CREATED' AND entity_id=?", (l["loan_id"],)).fetchone()[0] == 1
        for l in loans)
    checks["every loan has one instalment per month"] = all(conn.execute(
        "SELECT COUNT(*) FROM repayments WHERE loan_id=?", (l["loan_id"],)).fetchone()[0] == l["duration_months"] for l in loans)
    checks["outstanding balance matches settled instalments"] = True
    for l in loans:
        settled = conn.execute("SELECT COUNT(*) FROM repayments WHERE loan_id=? AND status IN ('paid','late')",
                               (l["loan_id"],)).fetchone()[0]
        schedule = ls.build_schedule(l["principal"], l["interest_rate"], l["duration_months"], ls._to_date(l["start_date"]))
        expected = l["principal"] if settled == 0 else schedule[settled - 1]["balance_after"]
        if l["status"] == "completed":
            expected = 0.0
        if abs(expected - l["outstanding_balance"]) > 0.01:
            checks["outstanding balance matches settled instalments"] = False
    checks["funded requests: loan principals add up to the request amount"] = all(
        abs(row["total"] - row["amount"]) < 0.01 for row in conn.execute(
            "SELECT SUM(lo.principal) AS total, r.amount FROM loans lo JOIN matches m ON m.match_id = lo.match_id "
            "JOIN loan_requests r ON r.request_id = m.request_id GROUP BY r.request_id"))
    checks["no lender has negative available funds"] = conn.execute(
```

```
        "SELECT COUNT(*) FROM lenders WHERE available_amount < 0").fetchone()[0] == 0
    checks["accepted matches == loans"] = conn.execute("SELECT COUNT(*) FROM matches WHERE status='accepted'").fetchone()[0] == len(loans)
    changed = conn.execute("SELECT COUNT(*) FROM repayments WHERE status != 'pending'").fetchone()[0]
    logged = conn.execute("SELECT COUNT(*) FROM audit_logs WHERE entity_type='repayment'").fetchone()[0]
    checks["every repayment status change is in the audit log"] = changed == logged
    checks["funded requests have loans"] = conn.execute(
        "SELECT COUNT(*) FROM loan_requests r WHERE r.status='funded' AND NOT EXISTS "
        "(SELECT 1 FROM matches m JOIN loans lo ON lo.match_id=m.match_id WHERE m.request_id=r.request_id)").fetchone()[0] == 0
    return checks


def evaluate(n_lenders, n_borrowers, seed=42, partial=False):
    path = os.path.join(tempfile.mkdtemp(), "eval.db")
    database.init_db(path)
    conn = database.get_connection(path)
    lenders, borrowers, requests = generate_data.generate(n_lenders, n_borrowers, seed)
    _, _, request_ids, _ = generate_data.load(conn, lenders, borrowers, requests)

    started = time.perf_counter()
    summary = MatchingEngine(conn, allow_partial=partial).run()
    match_seconds = time.perf_counter() - started

    violations, selected = count_constraint_violations(conn, partial)
    good, total = explanation_correctness(conn, partial)
    ls.accept_all(conn)
    ls.simulate_all(conn, "random", seed)
    checks = consistency_checks(conn)
    result = {
        "lenders": n_lenders, "requests": len(request_ids), "pairs_evaluated": n_lenders * len(request_ids),
        "match_seconds": round(match_seconds, 3), "eligible_matches": summary["eligible_edges"],
        "selected_matches": selected, "requests_matched": summary["requests_matched"],
        "allocation_success_rate": summary["allocation_success_rate"], "constraint_violations": violations,
        "explanation_correct": f"{good}/{total}", "explanation_correct_pct": round(100 * good / total, 2) if total else 100.0,
        "consistency_passed": f"{sum(checks.values())}/{len(checks)}", "consistency_details": checks,
        "loans_created": conn.execute("SELECT COUNT(*) FROM loans").fetchone()[0],
        "audit_entries": conn.execute("SELECT COUNT(*) FROM audit_logs").fetchone()[0]}
    conn.close()
    return result


def markdown_report(results, partial):
    lines = ["# P2P Lending Matchmaking - Evaluation Report", "",
             f"Funding mode: {'partial (co-funding)' if partial else 'full funding'}.  Simulated data only.", "",
             "| Lenders | Requests | Pairs | Match time (s) | Eligible | Selected | Matched | Success % |"
             " Violations | Explanations | Consistency |",
             "|---|---|---|---|---|---|---|---|---|---|---|"]
    for r in results:
        lines.append(f"| {r['lenders']} | {r['requests']} | {r['pairs_evaluated']} | {r['match_seconds']} | "
                     f"{r['eligible_matches']} | {r['selected_matches']} | {r['requests_matched']} | "
                     f"{r['allocation_success_rate']} | {r['constraint_violations']} | {r['explanation_correct']} | "
                     f"{r['consistency_passed']} |")
    lines += ["", "## Consistency checks (largest run)", ""]
    lines += [f"- [{'x' if ok else ' '}] {name}" for name, ok in results[-1]["consistency_details"].items()]
    return "\n".join(lines) + "\n"


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--sizes", default="10x15,50x80,200x300", help="lenders x borrowers, comma separated")
    parser.add_argument("--partial", action="store_true")
    parser.add_argument("--report", help="write a markdown report to this file")
    args = parser.parse_args()
    results = []
    for size in args.sizes.split(","):
        n_l, n_b = (int(x) for x in size.lower().split("x"))
        r = evaluate(n_l, n_b, partial=args.partial)
        results.append(r)
        print(f"\n=== {n_l} lenders x {n_b} borrowers ({r['requests']} requests, {r['pairs_evaluated']:,} pairs) ===")
        for key in ("match_seconds", "eligible_matches", "selected_matches", "requests_matched",
                    "allocation_success_rate", "constraint_violations", "explanation_correct",
                    "consistency_passed", "loans_created", "audit_entries"):
            print(f"  {key:<26} {r[key]}")
        failed = [k for k, ok in r["consistency_details"].items() if not ok]
        print("  consistency problems      ", failed or "none")
    if args.report:
        with open(args.report, "w", encoding="utf-8") as fh:
            fh.write(markdown_report(results, args.partial))
        print(f"\nReport written to {args.report}")


if __name__ == "__main__":
    main()
```

[NEW] *tests/test_week7.py*

```
"""Week 7 - the ten required test scenarios plus the evaluation metrics.

Run everything:   python -m pytest -v        (or)        python -m unittest discover -s tests -v
"""
import os
import sys
import unittest

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from helpers import add_borrower, add_lender, add_request, new_conn
import evaluate
import loan_service as ls
import profile_service as ps
import queries
from matching_engine import MatchingEngine
from models import ValidationError


def selected(conn):
    return [r for r in queries.list_pairs(conn, status="recommended") if r["allocated_amount"] > 0]


class RequiredScenarios(unittest.TestCase):
    def test_01_multiple_lenders_compete_for_one_borrower(self):
        conn, _ = new_conn()
        best = add_lender(conn, appetite="medium", min_ret=10, max_ret=15)
        add_lender(conn, appetite="high", min_ret=10, max_ret=15)
        add_lender(conn, appetite="high", min_ret=10, max_ret=15)
        add_request(conn, add_borrower(conn, income=45000, obligations=12000, credit=650, years=3), amount=60000, term=24)
        MatchingEngine(conn).run()
        rows = selected(conn)
        self.assertEqual([r["lender_id"] for r in rows], [best])
        self.assertEqual(len(queries.list_pairs(conn, status="expired")), 2)

    def test_02_one_lender_matched_with_multiple_loans(self):
        conn, _ = new_conn()
        add_lender(conn, available=500000)
        for _ in range(5):
            add_request(conn, add_borrower(conn), amount=40000)
        s = MatchingEngine(conn).run()
        self.assertEqual((s["requests_matched"], s["total_allocated"]), (5, 200000))

    def test_03_no_compatible_lender(self):
        conn, _ = new_conn()
        add_lender(conn, appetite="low", min_ret=1, max_ret=4)
        add_request(conn, add_borrower(conn))
        s = MatchingEngine(conn).run()
        self.assertEqual((s["eligible_edges"], s["requests_matched"], s["allocation_success_rate"]), (0, 0, 0.0))
        row = queries.list_pairs(conn)[0]
        self.assertEqual(row["status"], "rejected")
        self.assertIn("outside the lender's expected return", row["explanation"])

    def test_04_loan_amount_exceeds_available_funds(self):
        conn, _ = new_conn()
        add_lender(conn, available=25000)
        add_request(conn, add_borrower(conn), amount=60000)
        MatchingEngine(conn).run()
        self.assertIn("has 25,000 available but 60,000 is needed", queries.list_pairs(conn)[0]["explanation"])

    def test_05_risk_appetite_mismatch(self):
        conn, _ = new_conn()
        add_lender(conn, appetite="low", min_ret=6, max_ret=25)
        add_request(conn, add_borrower(conn, income=30000, obligations=5000, credit=520, years=1), amount=30000)
        MatchingEngine(conn).run()
        row = queries.list_pairs(conn)[0]
        self.assertEqual(row["status"], "rejected")
        self.assertIn("only accepts up to low risk", row["explanation"])

    def test_06_repayment_term_mismatch(self):
        conn, _ = new_conn()
        add_lender(conn, min_m=6, max_m=12)
        add_request(conn, add_borrower(conn), amount=30000, term=36)
        MatchingEngine(conn).run()
        self.assertIn("outside the lender's range 6-12 months", queries.list_pairs(conn)[0]["explanation"])

    def test_07_duplicate_loan_requests(self):
        conn, _ = new_conn()
        b = add_borrower(conn)
        add_request(conn, b, amount=30000, term=12)
        with self.assertRaises(ValidationError):
            add_request(conn, b, amount=30000, term=12)
        add_request(conn, b, amount=30000, term=18)                        # different term is fine
        self.assertEqual(len(ps.list_requests(conn)), 2)

    def test_08_large_datasets(self):
        r = evaluate.evaluate(60, 100)
        self.assertEqual(r["constraint_violations"], 0)
        self.assertLess(r["match_seconds"], 30)
        self.assertGreater(r["eligible_matches"], 0)

    def test_09_paid_late_and_defaulted_repayments(self):
        conn, _ = new_conn()
        add_lender(conn, available=500000)
        rids = [add_request(conn, add_borrower(conn), amount=30000, term=12) for _ in range(3)]
        MatchingEngine(conn).run()
        loans = [ls.accept_request(conn, rid)[0] for rid in rids]
        paid, late, default = (ls.simulate_loan(conn, lid, sc) for lid, sc in zip(loans, ("all_paid", "some_late", "default")))
        self.assertEqual((paid["status"], paid["outstanding_balance"]), ("completed", 0.0))
```

```
        self.assertEqual((late["status"], late["settled"]), ("completed", 12))
        self.assertEqual(default["status"], "defaulted")
        self.assertGreater(default["outstanding_balance"], 0)
        self.assertEqual(conn.execute("SELECT COUNT(*) FROM repayments WHERE status='late'").fetchone()[0], 3)

    def test_10_incorrect_or_missing_profile_information(self):
        conn, _ = new_conn()
        base = {"full_name": "A B", "email": "ab@x.com", "password": "password123", "monthly_income": 1000,
                "existing_obligations": 0, "credit_score": 600, "credit_history_years": 1}
        for bad in ({}, {**base, "credit_score": 1200}, {**base, "monthly_income": -5}, {**base, "email": "nope"},
                    {**base, "full_name": ""}, {**base, "credit_history_years": "abc"}, {**base, "password": "123"}):
            with self.assertRaises(ValidationError):
                ps.register_borrower(conn, bad)
        self.assertEqual(conn.execute("SELECT COUNT(*) FROM users").fetchone()[0], 0)      # nothing half-saved


class EvaluationMetrics(unittest.TestCase):
    def test_full_funding_metrics(self):
        r = evaluate.evaluate(30, 45)
        self.assertEqual(r["constraint_violations"], 0)
        self.assertEqual(r["explanation_correct_pct"], 100.0)
        self.assertTrue(all(r["consistency_details"].values()), r["consistency_details"])
        self.assertGreaterEqual(r["allocation_success_rate"], 0)

    def test_partial_funding_metrics(self):
        full, partial = evaluate.evaluate(30, 45), evaluate.evaluate(30, 45, partial=True)
        self.assertEqual(partial["constraint_violations"], 0)
        self.assertTrue(all(partial["consistency_details"].values()), partial["consistency_details"])
        self.assertGreaterEqual(partial["requests_matched"], full["requests_matched"])

    def test_runs_are_deterministic(self):
        a, b = evaluate.evaluate(20, 30, seed=5), evaluate.evaluate(20, 30, seed=5)
        for key in ("eligible_matches", "selected_matches", "requests_matched", "loans_created"):
            self.assertEqual(a[key], b[key])


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

[NEW] *README.md*

```
# Peer-to-Peer Lending Matchmaking System (Problem 105)

Graph-based risk-compatibility matching of lenders and borrowers.
Academic simulation using test data - no real money is moved.

## Setup (once)
    python -m venv venv
    venv\Scripts\activate            (Windows)   |   source venv/bin/activate   (macOS/Linux)
    pip install -r requirements.txt

## Run
    python app.py                    -> http://127.0.0.1:5000
    python demo_data.py              -> loads the demo dataset into an EMPTY database
    python demo_week2.py ... demo_week6.py   -> console demos (use their own demo databases)
    python generate_data.py --lenders 50 --borrowers 80 --out data/sample
    python evaluate.py --report test_report.md
    python -m pytest -v              (or)   python -m unittest discover -s tests -v

## Pipeline
profiles -> risk scoring -> compatibility scoring -> weighted bipartite graph (NetworkX)
-> match ranking -> loan simulation -> repayment tracking -> reports and audit

## Modules
| File | Purpose |
|---|---|
| config.py, schema.sql, database.py | settings, business rules, SQLite schema (Week 1) |
| models.py, profile_service.py, audit.py | OOP classes, validation, profile status (Week 2) |
| scoring.py, queries.py | risk score, hard constraints, compatibility score (Week 3) |
| matching_engine.py | bipartite graph, ranking, allocation, partial funding (Week 4) |
| loan_service.py | loans, schedules, repayment simulation (Week 5) |
| reports.py, routes/ | dashboards, REST API, CSV / Excel / PDF (Week 6) |
| generate_data.py, evaluate.py, tests/ | sample data, metrics, tests (Week 7) |
```

Final delivery checklist

Deliverable	Where it is
Source code	the whole project folder (Weeks 1-7)
Database	data/p2p_lending.db (created by python app.py or python database.py)
Sample dataset	data/sample/*.csv from generate_data.py, and demo_data.py
Test report	test_report.md + terminal output of pytest
Screenshots	take them from the lists at the end of every week
Project documentation	README.md and these PDFs
Presentation (PPT)	not included in these files - make 8-10 slides from the week summaries and screenshots
Working demonstration	python app.py, then: register profiles - Run scoring - Run matching - Accept all - Simulate repayments - /admin

Screenshots to attach to your submission

- Terminal with the full pytest run (all passed) and the evaluate.py output.
- test_report.md opened in VS Code.
- The final application: admin dashboard after the complete flow.